

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені Ігоря Сікорського»

Факультет прикладної математики

Кафедра прикладної математики

Звіт

з лабораторної роботи № 1

за темою «Реалізація каунтера з використанням PostgreSQL»

із дисципліни «Бази даних»

Виконала: Біловол Дарія

Мета

Необхідно декількома способами реалізувати оновлення значення каунтера в СКБД PostgreSQL та оцінити час кожного із варіантів.

Хід роботи

1. Створення таблиці

- У PostgreSQL створюється таблиця `user_counter` з полям:
 - `USER_ID` (ключ)
 - `Counter` (лічильник)
 - `Version` (для контролю версій)

2. Реалізація 4 підходів оновлення каунтера

- **Lost Update:** Потоки окремо читають і оновлюють `Counter`, що призводить до втрати даних.
- **In-Place Update:** Використовується `UPDATE counter = counter + 1`, що робить операцію атомарною.
- **Row-Level Locking:** `SELECT ... FOR UPDATE` блокує рядок, поки він не буде оновлений.
- **Optimistic Concurrency:** Використання `Version`, що запобігає одночасному зміні даних.

3. Запуск тестування продуктивності

- Кожен метод виконується в 10 потоках, кожен з яких робить 10 000 оновлень.
- Використовується `ThreadPoolExecutor`, щоб одночасно запускати оновлення.
- Вимірюється час виконання кожного методу.

4. Аналіз результатів

- **Lost Update** – швидкий, але призводить до втрати оновлень.
- **In-Place Update** – найшвидший, підходить, якщо важлива продуктивність.
- **Row-Level Locking** – захищає дані, але гальмує через блокування.
- **Optimistic Concurrency** – балансує між швидкістю та захистом від конфліктів.

Висновки

Для простого підрахунку лайків оптимальний `In-Place Update`. Якщо важлива точність, варто використовувати `Optimistic Concurrency` або `Row-Level Locking`.

Код

```
import psycopg2
from concurrent.futures import ThreadPoolExecutor
import time

# Налаштування підключення до бази даних
DB_CONFIG = {
    "dbname": "test_db_name",
    "user": "dari",
    "password": "dariismyname",
    "host": "localhost",
    "port": 5432
}

def lost_update():
    with psycopg2.connect(**DB_CONFIG) as conn:
        with conn.cursor() as cursor:
            cursor.execute("SELECT counter FROM user_counter WHERE user_id = 1")
            counter = cursor.fetchone()[0]
            counter += 1
            cursor.execute("UPDATE user_counter SET counter = %s WHERE user_id = 1", (counter,))
            conn.commit()

def in_place_update():
    with psycopg2.connect(**DB_CONFIG) as conn:
        with conn.cursor() as cursor:
            cursor.execute("UPDATE user_counter SET counter = counter + 1 WHERE user_id = 1")
            conn.commit()

def row_level_locking():
    with psycopg2.connect(**DB_CONFIG) as conn:
        with conn.cursor() as cursor:
            cursor.execute("SELECT counter FROM user_counter WHERE user_id = 1 FOR UPDATE")
            counter = cursor.fetchone()[0]
            counter += 1
            cursor.execute("UPDATE user_counter SET counter = %s WHERE user_id = 1", (counter,))
            conn.commit()

def optimistic_concurrency():
    while True:
        with psycopg2.connect(**DB_CONFIG) as conn:
            with conn.cursor() as cursor:
                cursor.execute("SELECT counter, version FROM user_counter WHERE user_id = 1")
                counter, version = cursor.fetchone()
                counter += 1
                cursor.execute("""
                    UPDATE user_counter
                """)
```

```

        SET counter = %s, version = %s
        WHERE user_id = 1 AND version = %s
        """, (counter, version + 1, version))
    conn.commit()
    if cursor.rowcount > 0:
        break

def measure_time(func, threads=10, iterations=10000):
    start_time = time.time()
    with ThreadPoolExecutor(max_workers=threads) as executor:
        for _ in range(threads):
            executor.submit(lambda: [func() for _ in range(iterations)])
    end_time = time.time()
    print(f"{func.__name__}: {end_time - start_time:.2f} seconds")

if __name__ == "__main__":
    print("Testing different approaches...")
    measure_time(lost_update)
    measure_time(in_place_update)
    measure_time(row_level_locking)
    measure_time(optimistic_concurrency)

```

Реалізація

```
Testing different approaches...  
lost_update: 0.19 seconds  
in_place_update: 0.18 seconds  
row_level_locking: 0.19 seconds  
optimistic_concurrency: 0.19 seconds
```